19. Bölüm
Ön İşlemci Yönergeleri

19.1 Ön İşlemci Yönergesi Nasıl Çalışır?

C dilinde olduğu gibi C++ dilinde de **ön işlemci** yönergeleri (preprocessor directive), derleyicinin bir parçası değildir, ancak derleme sürecinde ayrı bir adımdır. Ön işlemci, yalnızca bir metin değiştirme aracıdır ve gerçek derlemeden önce gerekli ön işlemeyi yapmasını sağlar. Derleme süeci, Şekil 19.1'de gösterilmiştir. Kısaca derleme önce bul-değiştir mantığı ile kodumuzda değişiklikler yapar.

- ◇ Ön işleme bir C kodunun derlenmesi öncesindeki ilk adımdır.
- ◇ Kodu belirteçlere ayırma (tokenization) adımından önce gerçekleşir.
- ◇ Ön işlemcinin önemli işlevlerinden biri de programda kullanılan kütüphane işlevlerini içeren başlık dosyalarını koda dahil etmek için kullanılmasıdır.
- ◇ Ön işlemci ayrıca değişmezleri (literal) tanımlar ve makro kullanımını sağlar.

Ön işlemci ifadelerine, **yönerge** (directive) denir. Programda ön işlemci bölümü her zaman C kodunun en üstünde görünür. Her ön işlemci ifadesi, kare (hash) (#) sembolüyle başlar. Aşağıda ön işlemci yönergelerinin listesi verilmiştir;

Yönerge	Açıklama
`#define`	Ön işlemci makrosunu değiştirmek ya da ilk defa tanımlamak için kullanılır.
`#include`	Bir başlık dosyasını diğer ile dahil etmek için kullanılır.
`#undef`	Tanımlanmış bir makroyu tanımsız hale getirmek için kullanılır.
`#ifdef`	Bir makro tanımlı ise doğru/true değerini döndürür.
`#ifndef`	Bir makro tanımlı değilse doğru/true değerini döndürür.
`#if`	Derleme zamanında bir durumun kontrolü için kullanılır.
`#else`	`#if` Yönergesinde alternatif durumu ifade eder.
`#elif`	`#else` ve `#if` yönergelerini tek bir şekilde ifade etmek için kullanılır.
`#endif`	Şart ön işlemcilerini (`#if`, `#else`, `#elif`) bitirir.
`#error`	stderr standart dosyasına hata mesajını yazar.
`#pragma`	Derleyiciye özel komutlar vermek için kullanılır.

Tablo 19.1: Ön İşlemci Yönergeleri

19.2 Ön işlemci Makroları

`#define` yönergesi ile yeni bir makro tanımlanabileceği gibi tanımlanmış bir makro `#undef` ile ortadan kaldırabilir veya yenisini tanımlayabiliriz. Aşağıdaki örnekte `MAX_STUDENTS` adlı bir makro tanımlanmış ve `100` tam sayısını vermektedir. Daha önce `MAX_STUDENTS` adlı bir makro tanımlanmış ise derleyici hata verir. Aşağıda buna bir örnek verilmiştir;

```
#include <iostream>

#ifndef MAX_STUDENTS
#define MAX_STUDENTS 100
#endif

int main() {
    std::cout << "Ilk MAX_STUDENTS: " << MAX_STUDENTS << std::endl;
#ifdef MAX_STUDENTS
#undef MAX_STUDENTS
#define MAX_STUDENTS 20
#endif
    std::cout << "Yeni MAX_STUDENTS: " << MAX_STUDENTS << std::endl;
}
```

Ön işlemci yönergeleriyle koşullu derleme yapılır. `#ifndef` (if not defined) veya `#ifdef` (if defined) ile kodun belirli bölümlerini duruma göre derleme dışı bırakabilir veya dahil edebilir.

Ön işlemci makrolarının (`#define`) veri tipi yoktur ve kapsam (scope) tanımazlar. Bu nedenle C++ dilinde makro kullanımı **sıkıntılıdır!**
Makroların, tip kontrolü yoktur. Bu nedenle hatalara açıktır.
Hata ayıklayıcılar (debugger) makroyu görmez. Bu nedenle makrolarda hata ayıklama zordur.
(Header guard) gibi koşullu derleme için makrolar kullanılmak zorundadır!

Modern C++'ta sabit tanımlamak için `constexpr` tercih edilir. Bu konu **??** başlığında işlenmiştir.

```
// sabitler için makro yerine:
#define PI 3.1415
// Modern C++ ta sabit ifadeler kullanılır:
constexpr double PI = 3.1415; // Tip güvenli ve derleme zamanı sabiti
```

19.3 Parametreli Ön İşlemci Makroları

Ön işlemci yönergeleriyle tanımlanan makrolar derlemeye geçilmeden önce işlem görür. Dolayısıyla bu aşamada bazen parametrelerle işlem yapılması gerekebilir.

```
#define kare(x) ((x) * (x))
#define kup(x) ((x) * (x) *(x))
#define buyugu(x,y) ((x) > (y) ? (x) : (y))
```

Burada kullanılacak parametreler veri tipi tanımlanmaz. Dolayısıyla kullanılacağı yerlere buna dikkat edilerek parametreli makro tanımlanmalıdır. Eğer makro birkaç satırdan oluşacak ise **ters bölü** (back slash) (\) karakteri kullanılır.

```
#include <iostream>
#define SAYIYAZ(num, str) {  \
    /* 1.Satir */     \
    std::cout << str;    \
    /* 2.Satir */     \
    std::cout << "=" << num << std::endl;  \
    /* Son satırda terbölü olmaz! */    \
    }
#define MY_MULTI_LINE_MACRO(arg1, arg2) do { \
    /* Line 1 of the macro */   \
    std::cout << "Arg1:" << arg1 << std::endl;  \
    /* Line 2 of the macro */    \
    std::cout << "Arg2:" << arg2 << std::endl;   \
    /* Last line (no backslash) */   \
    } while (0);
int main() {
    SAYIYAZ(4,"Dort")
    MY_MULTI_LINE_MACRO(1,2)
}
/*Program Çıktısı:
Dort=4
Arg1:1
Arg2:2
*/
```

C++ dilinde makro kullanımı sıkıntılıdır. Eski kodlarda karşılaşırsınız diye bu konu anlatılmıştır.

19.4 Ön Tanımlı Ön İşlemci Makroları

Her C derleyicisi için standart olarak tanımlı ön işlemci yönergesi ile makrolar bulunmaktadır;

Makro	Açıklama
`__DATE__`	Mevcut tarihi `"MMM DD YYYY"` biçiminde dizgi (string) olarak tanımlıdır.
`__TIME__`	Mevcut saat `"HH:MM:SS"` biçiminde dizgi (string) olarak tanımlıdır.
`__FILE__`	Dosya adı biçiminde dizgi (string) olarak tanımlıdır.
`__LINE__`	Dosyadaki satır numarası tam sayı (integer) olarak tanımlıdır.
`__STDC__`	ANSI standardında derleme yapılıyorsa 1 olarak tanımlıdır.

Tablo 19.2: Standart Ön İşlemci Makroları

Standart makroları kullanan örnek program aşağıda verilmiştir;

```
#include <iostream>
int main() {
    std::cout << __FILE__ << std::endl; //File: main.c
    std::cout << __DATE__ << std::endl; //Date: Mar 22 2025
    std::cout << __TIME__ << std::endl; //Time: 15:32:08
    std::cout << __LINE__ << std::endl; //Line: 6
    std::cout << __STDC__ << std::endl; //ANSI: 1
}
```

Bu makrolar, özellikle büyük projelerde hata ayıklama (debug) ve iz kaydı (log) tutma işlemleri için hayat kurtarıcıdır. Modern C++ dünyasında bu bilgiler hâlâ aynı isimlerle kullanılır, ancak çıktı yöntemleri ve standartlar biraz daha gelişmiştir.

Modern C++ (C++11 ve sonrası) bu makrolara ek olarak `__func__` (veya derleyiciye göre `__FUNCTION__`) makrosunu da getirmiştir. Bu makro, hatanın hangi fonksiyonun içinde gerçekleştiğini anlamamızı sağlar.

```
#include <iostream>
void hataLogu(const std::string& mesaj, const char* dosya, int satir, const char* fonk) {
    std::cerr << "[HATA] " << dosya << ":" << satir << " (" << fonk << ") -> " << mesaj <<
    ↪ std::endl;
}
int main() {
    // Örnek bir hata simülasyonu
    hataLogu("Veritabanı baglantisi koptu!", __FILE__, __LINE__, __func__);
}
```

Eğer C++20 kullanıyorsanız, bu makroların yerine çok daha nesne yönelimli ve "temiz" bir yol olan `std::source_location` sınıfını kullanabilirsiniz:

```
#include <iostream>
#include <source_location>
void bilgiYazdir(const std::source_location location = std::source_location::current()) {
    std::cout << "Dosya: " << location.file_name() << "\n" << "Fonksiyon: " <<
    ↪ location.function_name() << "\n" << "Satir: " << location.line() << std::endl;
}
int main() {
    bilgiYazdir();
}
```

19.5 Başlık Koruması

Ön işlemci direktiflerinin en yaygın ve profesyonel kullanım alanı başlık (header) koruması (header guard) yapısıdır. Bir başlık dosyasının (.h) kazara birden fazla kez projeye dahil edilip yeniden tanımlama (redefinition) hatası vermesini engeller.

```
// Ogrenci.h dosyası içeriği:
#ifndef OGRENCI_H  // Eğer OGRENCI_H tanımlı değilse
#define OGRENCI_H  // OGRENCI_H tanımla

class Ogrenci {
    // Sınıf tanımları
};

#endif // OGRENCI_H sonu
```

Başlık dosyasının bir koda birden fazla dahil (include) edilmesini önlemek için `OGRENCI_H` makrosu, şartlı (conditional) olarak tanımlanmıştır. Bu başlık dosyası bir kod projesinde birden fazla dahil olması halinde, `OGRENCI_H` tanımlanmaz ise, başlık içindeki değişken ve fonksiyonlar birden fazla aynı kimlikle tanımlanacağından derleme hatası alınacaktı.

Kendi hazırlamış olduğumuz `Ogrenci.h` başlık dosyasını kodumuza dahil ettiğimizde artık başlık dosyamızdaki fonksiyon ve değişkenleri kullanabilir hale geliriz. Aynı klasörde/dizinde bulunacak şekilde bu başlık dosyasını kullanan `main.c` örnek programı aşağıda verilmiştir;

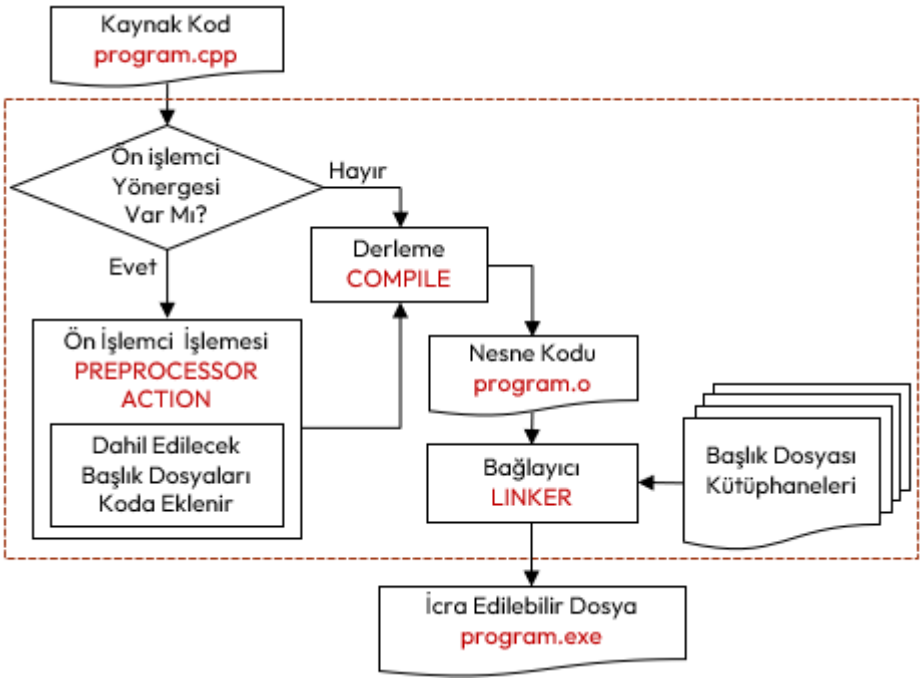
```
#include <iosreram>
#include "Ogrenci.h"
```

```
int main() {
    Ogrenci ali;
    //...
}
```

19.6 Derleme Sürecinin Detayı

C++ dilinde derleme sürecinin iki aşamadan oluştuğu daha önce anlatılmıştı. Aşağıda tüm süreç verilmiştir;

1. Birinci aşama:
   (a) Kaynak koddaki tüm açıklamalar silinir.
   (b) `#define`, `#if`, `#include`, gibi tüm yönergeler ile verilen işlemler yapılır.
   (c) Bunlar arasında başlık dosyalarının koda dahil edilmesi de vardır.
   (d) Bu duruma gelen kod, `g++ -E program.cpp` komutu ile dosya haline getirilebilir.
2. İkinci aşama:
   (a) Kaynak kod montaj koda (assembly code) çevrilir.
   (b) Bu duruma gelen kod, `g++c -S program.cpp` komutu dosya haline getirilebilir.
   (c) Assembly kod derlenerek makine diline çevrilir ve amaç dosya (object file) oluşur.
   (d) Bu duruma gelen kod, `g++ -c program.cpp` komutu ile dosya haline getirilebilir.
   (e) Amaç dosya kodda belirtilen başlık dosyalarına uygun kütüphaneler (library) ile birbirine bağlayıcı (linker) ile bağlanır ve icra edilebilir dosya (executable file) elde edilir. Burada bahsedilen kütüphaneler her işletim sistemi için ayrı olarak oluşturulur ve derleyici kurulumunda bir klasörde tutulur.
   (f) Bu aşamaya gelen derleme `g++ program.cpp -o program.exe` komutuyla icra edilebilir dosya oluşturulmaktadır.



Şekil 19.1: Derleme Süreci

19.7 Düşün ve Çöz

- **Düşün**: `#define` ile tanımlanan makroların, normal fonksiyonlara göre avantajları ve riskleri (yan etkileri) nelerdir?
- **Düşün**: Başlık dosyalarında kullanılan `#ifndef`, `#define`, `#endif` **başlık güvenliği** (header guard) bloklarının mükerrer dahil etmeyi nasıl önlediğini açıklayınız.
- **Düşün**: Koşullu derleme (`#ifdef DEBUG`) kullanarak, programın sadece geliştirme aşamasında ekrana teknik loglar basmasını nasıl sağlarsınız?
- **Çöz**: Bir sayının karesini alan bir makro (`#define KARE(x)`) tanımlayınız.`KARE(3+2)` ifadesinin sonucunu makronun nasıl açıldığını düşünerek analiz ediniz.
- **Çöz**: İki sayının küçüğünü bulan bir fonksiyonel makro (`#define MIN(a,b)`) yazınız ve makro **yan etkilerini** (side effects) önlemek için neden parantez kullanılması gerektiğini gösteriniz.

◇ **Çöz**: C'nin ön tanımlı makrolarını (`__DATE__`, `__TIME__`, `__FILE__`) kullanarak programın derlendiği anı ve dosya adını ekrana basan bir "Sürüm Bilgisi" fonksiyonu yazınız.